CoreAxis Technologies

Software Engineering Department Handbook

Engineering Standards, Practices, and Procedures

Section 1

Welcome to the Software Engineering Department

The Software Engineering Department is responsible for designing, developing, testing, deploying, and maintaining the software platforms that power CoreAxis Technologies' products and services. This handbook establishes the engineering standards, development practices, architectural principles, and operational procedures that all engineering personnel are expected to follow.

Engineering teams collaborate closely with the AI & Machine Learning, Information Technology, Information Security, Finance, and Human Resources departments to deliver secure, scalable, and maintainable software solutions. Our engineering department is responsible for designing, building, and maintaining the software systems that power these products.

Purpose of This Handbook

This handbook serves three primary functions:

- **Alignment:** Establishes common standards and expectations across all engineering teams, ensuring consistency in how we design, build, and ship software.
- **Onboarding:** Accelerates the integration of new engineers by providing a single, authoritative source of truth for how we operate.
- **Governance:** Defines engineering policies that reduce risk, improve reliability, and ensure our software meets the highest standards of quality and security.

How to Use This Document

This handbook is organised into logical sections that progress from team structure and process to technical standards and operational procedures. Engineers are encouraged to read sections relevant to their immediate role and responsibilities. Engineering Managers and Technical Leads are expected to be familiar with all sections.

This document is a living artifact. It is maintained by Engineering Leadership and updated on a quarterly basis, or whenever significant policy changes occur. The version and effective date are printed in the document header. Engineers are responsible for staying current with the latest revision.

Policy: This handbook is classified as internal use only. Do not distribute any part of this document outside CoreAxis Technologies without explicit written approval from Engineering Leadership.

Contact and Feedback

Questions about the standards described in this handbook should be directed to your Engineering Manager or raised in the `#engineering-standards` Slack channel. Proposed amendments must be submitted as a Confluence page draft tagged with the

label `handbook-proposal` for review by Engineering Leadership before inclusion in a subsequent revision.

Section 2

Department Overview

The Software Engineering Department at CoreAxis Technologies is responsible for the end-to-end delivery of software products and platform infrastructure. The department operates out of our Chennai headquarters and Bengaluru branch office, following a hybrid working model. All engineers are expected to be present in office on agreed anchor days as communicated by their Engineering Manager.

Working Hours and Availability

Standard working hours are Monday through Friday, 09:00 to 17:00 IST. Engineers are expected to be available and responsive during core hours. Flexible arrangements outside these hours must be agreed upon with the Engineering Manager and must not compromise sprint commitments or on-call obligations.

Departmental Mission

The Software Engineering Department exists to translate product vision into reliable, scalable, and secure software systems. Our department aligns directly with the company mission: to build secure, scalable, reliable, and responsible AI solutions that help businesses automate workflows and improve decision-making.

Relationship with Other Departments

Department	Primary Interaction
AI & Machine Learning	Model integration, LLM API consumption, RAG pipeline design, prompt engineering collaboration
Information Technology	Infrastructure provisioning, network configuration, workstation support, VPN and access management
Information Security	Security reviews, penetration testing coordination, vulnerability management, compliance requirements
Human Resources	Recruitment, performance management, training and development, policy compliance
Finance	Cloud cost governance, vendor contract management, budget approvals for tooling

Engineering Locations

The department operates across two offices:

- **Chennai, Tamil Nadu (Head Office):** Primary engineering hub. Engineering Leadership, core platform teams, and AI integration teams are based here.
- **Bengaluru, Karnataka (Branch Office):** Product engineering teams and DevOps function.

Cross-location collaboration is conducted through asynchronous tools (Jira, Confluence, GitHub Enterprise) and synchronous meetings scheduled in IST. Engineers in both locations are expected to maintain the same standards as described in this handbook.

Team Size and Growth

The engineering organisation currently supports approximately 100 employees across all departments. The engineering department constitutes a significant portion of this headcount. Growth is planned and managed through quarterly headcount reviews in coordination with HR and Finance.

Section 3

Engineering Team Structure

Roles, Responsibilities, and Expectations

The Software Engineering Department is structured around functional roles. Each role carries a defined set of responsibilities and expectations. Engineers may operate across multiple areas depending on project requirements, but primary accountability is always clearly assigned.

Backend Engineer

Backend Engineers design, develop, and maintain server-side systems, APIs, and data pipelines. They are responsible for the reliability, performance, and scalability of backend services.

- Design and implement RESTful APIs following the standards in Section 12.
- Write and maintain server-side application logic in Python (FastAPI/Flask) or Node.js.
- Design database schemas and manage migrations (PostgreSQL, MongoDB).
- Integrate with AI services, LLM APIs, and vector databases as defined by the AI & ML team.

- Ensure backend services meet defined SLAs for availability and response time.
- Write unit and integration tests covering all business logic.

Frontend Engineer

Frontend Engineers build user interfaces and client-side application logic. They are responsible for the usability, performance, and accessibility of all user-facing products.

- Build responsive, accessible web interfaces using React, TypeScript, HTML, and CSS.
- Consume and integrate APIs provided by backend teams.
- Ensure cross-browser compatibility and mobile responsiveness.
- Write end-to-end tests for critical user journeys using Playwright.
- Optimise rendering performance and Core Web Vitals metrics.
- Coordinate with product and design stakeholders on UI/UX implementation.

Full Stack Engineer

Full Stack Engineers operate across the entire application stack. They are expected to maintain backend engineering standards at the same level as Backend Engineers and frontend standards at the same level as Frontend Engineers.

- Own features from API design through to UI delivery.
- Collaborate with both backend and frontend specialists on complex integrations.
- Make pragmatic architectural trade-offs and document decisions in Confluence.

DevOps Engineer

DevOps Engineers manage the infrastructure, CI/CD pipelines, container orchestration, and monitoring systems that underpin all software delivery.

- Design and maintain CI/CD pipelines using GitHub Actions.
- Manage Docker container definitions and Kubernetes deployment manifests.
- Provision and maintain cloud infrastructure on AWS and Azure.
- Operate and tune monitoring, alerting, and observability tooling (Prometheus, Grafana, ELK Stack).
- Enforce infrastructure-as-code practices for all environment changes.
- Conduct post-incident reviews and maintain the incident runbook.

QA Engineer

QA Engineers define test strategies, write automated test suites, and act as the quality gate for all software releases.

- Maintain the test strategy document for each product area.
- Write and maintain automated test suites using Pytest, Postman, and Playwright.
- Execute exploratory testing sessions prior to major releases.
- File and triage defect reports in Jira with appropriate severity classifications.
- Validate that acceptance criteria are satisfied before marking stories as done.
- Track quality metrics across sprints and report regressions to Engineering Leadership.

Engineering Manager

Engineering Managers are accountable for the performance, wellbeing, and delivery outcomes of their engineering teams. They operate at the intersection of engineering and product management.

- Conduct weekly 1:1 meetings with all direct reports.
- Own sprint planning, retrospectives, and team-level capacity management.
- Escalate technical risks and blockers to Engineering Leadership.

- Enforce adherence to engineering standards defined in this handbook.
- Partner with HR on recruitment, performance reviews, and career development.
- Review and approve major architectural decisions within their product area.

Technical Lead

Technical Leads are senior engineers responsible for the technical direction of one or more product areas. They are hands-on contributors who also guide and review the work of other engineers.

- Own Architecture Decision Records (ADRs) for their product area.
- Perform final code review approvals for significant changes.
- Mentor mid-level and junior engineers through pairing and structured feedback.
- Represent the team in cross-department technical discussions.
- Evaluate new tooling, frameworks, and architectural patterns for suitability.

Section 4

Software Development Lifecycle (SDLC)

CoreAxis Technologies follows a structured Software Development Lifecycle that governs how software moves from concept to production. The SDLC defines the stages, artefacts, and quality gates that every software feature or system change must pass through before reaching production.

SDLC Stages

1. Requirements and Planning

All software work begins with clearly defined requirements. Product Managers, Technical Leads, and Engineering Managers collaborate to produce user stories, acceptance criteria, and technical specifications. Requirements are captured in Jira and linked to the relevant Epic. Ambiguous requirements must be resolved before an item enters sprint planning — the engineering team is not expected to begin implementation on incomplete specifications.

2. System Design

For any feature or change that materially affects system architecture, data schemas, or API contracts, a technical design document must be produced in Confluence before implementation begins. The design document must address: system context, data flow, API contracts, data schema changes, security considerations, and deployment implications. Significant designs require review and sign-off from the Technical Lead.

3. Development

Development follows the coding standards, Git workflow, and branching strategy defined in Sections 7–11. Engineers work on short-lived feature branches, commit frequently, and raise Pull Requests for peer review. No code is merged to the main branch without satisfying the PR and code review standards defined in this handbook.

4. Code Review

All code changes undergo peer review as defined in Section 11. The review process validates correctness, adherence to standards, test coverage, and security posture. Automated checks (linting, type checking, unit tests, security scanning) must pass before human review begins.

5. Testing

Testing is not a phase that occurs after development — it is a continuous activity embedded in every stage. Unit tests are written alongside the code. Integration tests are run in CI. End-to-end tests validate critical user journeys. QA Engineers conduct exploratory testing before release. No release proceeds without all defined quality gates passing.

6. Deployment

Deployment to production is managed through the CI/CD pipeline as defined in Section 26. All deployments are version-controlled, automated, and reversible. Manual production deployments are prohibited. The release manager coordinates deployment windows and communicates status to stakeholders.

7. Monitoring and Maintenance

Post-deployment, all systems are monitored through Prometheus, Grafana, and the ELK Stack. Alerting thresholds are pre-configured for each service. On-call engineers are responsible for responding to production incidents within defined SLA windows. Maintenance activities including dependency updates and technical debt reduction are planned into each sprint.

SDLC Artefacts

Stage	Required Artefact	Owner
Requirements	User Stories + Acceptance Criteria in Jira	Product Manager
Design	Technical Design Document in Confluence	Technical Lead

Development	Feature branch, commits, Pull Request	Engineer
Review	PR review comments, approval	Peer reviewers + Technical Lead
Testing	Test results, defect reports in Jira	QA Engineer
Deployment	Release notes, deployment record	DevOps + Release Manager
Monitoring	Dashboards, alert configurations, incident records	

Section 5

Agile Development Process

CoreAxis Technologies engineering teams operate under an Agile framework built around two-week Sprints. All software engineering teams follow this framework unless a documented exception has been approved by Engineering Leadership.

Sprint Cadence

Each sprint is two weeks in duration. The sprint begins on Monday and closes on the following Friday. The standard sprint ceremonies are:

Ceremony	Timing	Duration	Owner

Sprint Planning	Monday, Sprint Day 1	2 hours	Engineering Manager
Daily Standup	Daily, 09:30 IST	15 minutes	Team (rotating facilitator)
Sprint Review / Demo	Friday, Sprint Day 10	1 hour	Engineering Manager + Product
Sprint Retrospective	Friday, Sprint Day 10	1 hour	Engineering Manager
Backlog Refinement	Wednesday, Sprint Day 7	1 hour	Engineering Manager + Tech Lead

Story Points and Estimation

Teams use the Fibonacci sequence (1, 2, 3, 5, 8, 13) for story point estimation during Planning Poker sessions. A story point represents relative effort, not calendar time. Stories estimated at 13 points must be decomposed into smaller stories before they enter a sprint. Stories above 8 points should be reviewed with the Technical Lead for decomposition opportunities.

Definition of Ready

A Jira story is considered Ready for sprint planning when it satisfies all of the following:

- Acceptance criteria are written and agreed upon by product and engineering.
- Dependencies are identified and either resolved or explicitly accepted as risks.
- The story has been estimated in a refinement session.
- Any required technical design is completed or scoped as a task within the story.
- UI mockups or data contracts are available where applicable.

Definition of Done

A story is Done only when all of the following are satisfied:

- All acceptance criteria are met and verified by a QA Engineer or the story owner.
- Code is merged to the main branch via an approved Pull Request.
- All CI checks pass: linting, type checking, unit tests, integration tests.
- Unit test coverage meets the minimum threshold defined in Section 22.
- API or schema changes are reflected in updated documentation.
- The feature is deployed to the staging environment and functionally verified.
- No high or critical severity bugs remain open against the story.

Sprint Velocity and Capacity

Engineering Managers track velocity as a trailing average over the last three sprints. Velocity is used only for planning purposes — it is not a performance metric for individual engineers. Capacity planning accounts for leave, meetings, and maintenance overhead. A standard two-week sprint with a five-person team typically allocates 60–70% of total hours to story delivery, with the remainder covering ceremonies, code review, and unplanned work.

Unplanned Work

Unplanned work (production incidents, urgent bug fixes, security patches) is tracked in Jira under the label `unplanned`. Engineering Managers report unplanned work volume at each retrospective to identify systemic issues. Chronic unplanned work that exceeds 20% of sprint capacity triggers a root-cause discussion with Engineering Leadership.

Project Planning and Sprint Management

Project planning at CoreAxis Technologies bridges the gap between strategic product objectives and sprint-level delivery. Engineering Managers, Technical Leads, and Product Managers collaborate on quarterly planning cycles that inform the backlog for individual teams.

Quarterly Planning

At the start of each quarter, Engineering Leadership conducts a planning session with Engineering Managers and Technical Leads to align on OKRs (Objectives and Key Results), prioritise epics across product areas, and assess resourcing. Outputs of quarterly planning are captured in Confluence under the project's Planning page and linked to Jira epics.

Epic and Story Hierarchy

Work is organised using the following hierarchy in Jira:

- **Initiative:** A strategic objective spanning multiple quarters. Owned by Engineering Leadership or Product.
- **Epic:** A body of work deliverable within one to two quarters. Owned by the Engineering Manager and Technical Lead.
- **Story:** A user-facing unit of value completable within a single sprint. Owned by an individual engineer.
- **Sub-task / Task:** A technical decomposition of a story. Used for parallel work within a story.

- **Bug:** A defect reported against an existing feature. Triaged by the QA Engineer and prioritised by the Engineering Manager.

Backlog Management

The product backlog is maintained by the Product Manager in Jira, with technical input from the Technical Lead. The backlog must be groomed to a minimum of two sprints ahead at all times. Stories in the top two sprints of the backlog must be in Ready state (see Definition of Ready in Section 5).

Risk Management

Technical risks identified during planning or development are logged in the Confluence risk register for the relevant project. Each risk entry includes: risk description, probability, impact, mitigation strategy, and owner. Risks rated as High probability × High impact are escalated to Engineering Leadership within 48 hours of identification.

Stakeholder Communication

Engineering Managers publish a brief Sprint Summary to stakeholders at the end of each sprint. The summary includes: stories delivered, stories not completed (with reason), velocity, and any risks or blockers for the next sprint. This communication is distributed through the relevant Slack project channel and archived in Confluence.

Capacity Tracking

Engineering Managers maintain a capacity tracker in Confluence for their team, updated at sprint start. The tracker records: team members' available days, booked leave, planned meetings overhead, and resulting effective engineering days. Sprint commitments are calibrated against tracked capacity, not against a fixed theoretical maximum.

Section 7

Coding Standards

CoreAxis Technologies enforces a consistent set of coding standards across all production codebases. These standards improve readability, reduce review friction, and make onboarding faster. Adherence to these standards is enforced through automated tooling in the CI pipeline — non-compliant code will not be merged.

General Principles

- **Clarity over cleverness.** Write code that the next engineer can understand without asking you for help. Optimise for readability, not brevity.
- **SOLID principles.** Apply Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion consistently.
- **Clean Architecture.** Separate concerns clearly: business logic must not depend on infrastructure details. Domain logic is framework-agnostic.
- **No magic numbers.** Replace literal values with named constants or configuration entries.
- **No dead code.** Remove commented-out code and unused imports before raising a PR.
- **Fail fast.** Validate inputs at system boundaries. Surface errors early with meaningful messages.

Language-Specific Standards

Python

- Follow PEP 8 for style. Use `ruff` for linting and formatting.

- Type annotations are mandatory on all function signatures (parameters and return types).
- Use `pydantic` v2 models for data validation. Do not use raw dictionaries as function parameters across module boundaries.
- Async functions must use `async def`; do not mix synchronous blocking calls inside async contexts.
- Use `__slots__` on performance-sensitive classes.
- Target Python 3.11 or newer.

TypeScript / JavaScript

- Strict TypeScript mode (`strict: true`) is mandatory. No use of `any` without a comment explaining why it is necessary.
- Use ESLint with the team's shared configuration (`@coreaxis/eslint-config`) and Prettier for formatting.
- Prefer `const` over `let`. Never use `var`.
- Use async/await over raw Promise chains for readability.
- Named exports are preferred over default exports in shared modules.
- Target ES2022+ and Node.js LTS for server-side code.

Formatting

Language	Formatter	Indentation	Max Line Length
Python	ruff format	4 spaces	100 characters
TypeScript/JavaScript	Prettier	2 spaces	100 characters
Java	Google Java Format	2 spaces	120 characters

YAML / JSON	Prettier	2 spaces	—

Naming Conventions

Construct	Python	TypeScript
Variables / Functions	snake_case	camelCase
Classes / Types / Interfaces	PascalCase	PascalCase
Constants	UPPER_SNAKE_CASE	UPPER_SNAKE_CASE
Modules / Files	snake_case.py	kebab-case.ts
Database columns	snake_case	snake_case
Environment variables	UPPER_SNAKE_CASE	UPPER_SNAKE_CASE

Comments and Documentation

Code comments explain *why*, not *what*. Code should be self-documenting in expressing what it does. Comments that simply restate the code are noise and must be removed during review. All public API functions, classes, and modules require docstrings (Python) or JSDoc comments (TypeScript). Private helpers do not require documentation unless the logic is non-obvious.

Section 8

Git and Version Control Standards

All source code at CoreAxis Technologies is managed in Git repositories hosted on GitHub Enterprise. Version control standards govern how engineers interact with the repository to ensure a clean, auditable history and a reliable deployment pipeline.

Repository Conventions

- Repository names use `kebab-case`.
- Every repository must have a `README.md` that describes the service purpose, local development setup, and environment variable reference.
- Every repository must have a `.gitignore` appropriate for the language and framework.
- Secrets, credentials, and API keys are never committed to any repository, including private ones. See Section 20 for secret management policy.
- Binary files and generated build artefacts are not committed. Use `.gitignore` to exclude them and artefact registries (AWS ECR, GitHub Packages) for build outputs.

Commit Message Standards

All commit messages must follow the Conventional Commits specification. This enables automated changelog generation and semantic release tooling.

Format: `<type>(<scope>): <description>`

Type	When to Use
`feat`	A new feature visible to users or consumers of the API
`fix`	A bug fix

`refactor`	Code restructuring with no behaviour change
`test`	Adding or modifying tests
`docs`	Documentation changes only
`chore`	Build system, dependency updates, CI changes
`perf`	Performance improvement
`security`	Security fix or hardening change
`ci`	Changes to CI/CD configuration

Examples of acceptable commit messages:

```
feat(auth): add JWT refresh token endpoint
```

```
fix(ingestion): handle empty document list in RAG pipeline
```

```
chore(deps): upgrade langchain to 0.2.15
```

```
security(api): enforce rate limiting on public inference endpoints
```

Commit Hygiene

- Commits should be atomic — each commit represents a single coherent change.
- Do not commit changes that break the build or tests.
- Do not include unrelated changes in the same commit.
- Commit messages must be written in the imperative mood: "add feature" not "added feature".

- Interactive rebase (`git rebase -i`) may be used to clean up local commits before raising a PR. Do not rebase commits that have already been pushed to a shared branch.

Protected Branches

The following branch protection rules apply to all repositories:

- `main` and `production` branches require at least two approving reviews on any PR.
- Force pushes to protected branches are disabled.
- All CI checks must pass before a PR can be merged.
- Branch deletion after merge is enabled and enforced to keep the repository clean.

Section 9

Branching Strategy

CoreAxis Technologies uses a trunk-based branching strategy with short-lived feature branches. This approach minimises merge conflicts, encourages frequent integration, and keeps the main branch always deployable.

Branch Model

main

The `main` branch is the single source of truth for the production-ready codebase. It must always be in a deployable state. No direct commits to `main` are permitted. All

changes reach `main` through Pull Requests that pass all required checks and receive the required number of approvals.

Feature Branches

Feature branches are created from `main` and merged back to `main` via Pull Request. Branch names follow this convention:

```
<type>/<jira-ticket>-<short-description>
```

```
Examples:
```

```
feat/CAX-1234-rag-document-chunking
```

```
fix/CAX-1289-null-pointer-in-embedding-service
```

```
chore/CAX-1301-upgrade-fastapi-to-0-110
```

Feature branches must be short-lived. The target lifetime is one to three days. Branches that remain open for more than seven calendar days without activity are flagged for review by the Engineering Manager.

Release Branches

For products that require controlled releases (e.g. enterprise on-premise deployments), a `release/vX.Y` branch is created from `main` at the point of release cut. Hotfixes to a released version are cherry-picked to the appropriate release branch. Release branch creation requires approval from the Technical Lead and the Engineering Manager.

Hotfix Branches

Production hotfixes that cannot wait for the normal sprint cycle follow this workflow:

- Branch from `main` with the prefix `hotfix/` and the Jira ticket reference.
- The PR requires approval from at least one Technical Lead or Engineering Manager.
- After merging to `main`, cherry-pick the fix to any active release branches if applicable.
- A postmortem or incident review must be created in Confluence if the hotfix addresses a production outage.

Branch Lifetime Policy

Branch Type	Maximum Lifetime	Action on Expiry
Feature branch	7 calendar days	Review and close or merge
Hotfix branch	48 hours	Must merge or escalate to EM
Release branch	Duration of release support window	Archive when EOL

Integration Hygiene

Engineers must rebase their feature branch on `main` before raising a PR to ensure the change applies cleanly. Merge commits from `main` into feature branches are discouraged — use `git rebase origin/main` instead. This keeps the history linear and readable.

Policy: All merges to `main` use squash merge. This ensures a clean, linear commit history on `main` and makes rollbacks straightforward. The squash commit message must conform to the Conventional Commits standard (Section 8).

Section 10

Pull Request Guidelines

Pull Requests (PRs) are the primary mechanism for introducing changes to shared codebases. A well-structured PR accelerates review, reduces back-and-forth, and documents the intent behind changes. Engineers are expected to treat the PR description as important engineering communication, not a formality.

PR Size

PRs should be small and focused. The recommended maximum is 400 lines of changed code (additions plus deletions) per PR. Larger PRs are harder to review thoroughly and correlate with higher defect rates. If a feature requires more than 400 lines of change, decompose it into a series of smaller PRs that can each be reviewed and merged independently. The first PR in such a sequence may establish interfaces or data models; subsequent PRs implement logic behind those contracts.

PR Title

The PR title must follow the same Conventional Commits format as commit messages (Section 8). It becomes the squash merge commit message on `main`.

PR Description Template

Every PR must include a description that covers the following sections:

```
## Summary
What does this PR change, and why?
```

```
## Related Jira Ticket
[CAX-XXXX](https://coreaxis.atlassian.net/browse/CAX-XXXX)

## Type of Change
- [ ] New feature
- [ ] Bug fix
- [ ] Refactor
- [ ] Infrastructure / CI change
- [ ] Documentation

## Testing
How was this change tested? What automated tests were added or updated?

## Deployment Notes
Are there any migration steps, environment variable changes, or feature flag updates required when deploying this change?

## Screenshots / Recordings (if UI change)
```

Draft PRs

Draft PRs may be used to share work-in-progress code for early feedback. Draft PRs should not be assigned to reviewers — instead, tag colleagues in the PR description or Slack to request targeted input. A PR must be moved out of Draft status before the formal review process begins.

Self-Review

Before requesting review, the author must conduct a self-review of the PR diff. The author should verify: no debug code or temporary logging is present, no secrets or credentials are included, all files changed are intentional, and the PR description accurately reflects the changes made.

Linking Work Items

Every PR must reference its associated Jira ticket. Use the Jira Smart Commits syntax in the PR description or commit messages to automatically transition Jira issues:

```
CAX-1234 #in-review
```

Merge Requirements

- All CI checks (lint, type check, tests, security scan) must pass.
- A minimum of two approvals from eligible reviewers (see Section 11).
- No unresolved review comments.
- The branch must be up-to-date with `main` (rebased, not merged).

Section 11

Code Review Standards

Code review at CoreAxis Technologies is a collaborative quality process, not a gatekeeping exercise. The goal is to improve the code and share knowledge across the team. Reviewers and authors are expected to engage constructively and professionally.

Reviewer Eligibility

Any engineer with repository access may review a PR. However, the following rules apply to approvals that count toward the merge requirement:

- At least one approving reviewer must be a different engineer than the PR author.
- For changes that affect core business logic, data schema, or API contracts, at least one approval must come from the Technical Lead or a Senior Engineer.
- Engineers may not approve their own PRs.

Review Turnaround

Reviewers are expected to provide initial feedback within one business day of being assigned. If a reviewer cannot complete the review within this window, they should communicate this in the PR comments and the author may request a substitute reviewer. Stale PRs (no activity for three or more business days) are escalated to the Engineering Manager.

What Reviewers Check

Reviewers evaluate the following dimensions:

Correctness

Does the code correctly implement the stated requirements? Does it handle edge cases, null values, empty collections, and error conditions? Do unit tests adequately verify the behaviour?

Design

Is the abstraction level appropriate? Does the code follow SOLID principles and Clean Architecture guidelines? Are responsibilities correctly separated? Is there unnecessary coupling introduced?

Security

Are inputs validated and sanitised at system boundaries? Are SQL queries parameterised? Is authentication and authorisation correctly enforced? Are secrets handled according to policy?

Performance

Are there obvious N+1 query problems? Are expensive operations appropriately cached? Is there unnecessary work done in hot paths?

Observability

Are appropriate log statements added? Are errors surfaced with sufficient context? Are new metrics or traces added for significant new operations?

Standards Adherence

Does the code conform to the coding standards in Section 7? Are naming conventions followed? Are comments appropriate and correct?

Review Comment Protocol

Review comments use the following prefix convention:

Prefix	Meaning
`[blocking]`	Must be resolved before approval. Represents a correctness issue, security flaw, or standards violation.
`[suggestion]`	A recommended improvement. Not blocking. Author should consider and respond.
`[nit]`	A minor stylistic preference. Non-blocking. Author may address at discretion.
`[question]`	A clarifying question. Non-blocking; the answer may lead to further discussion.

Author Responsibilities During Review

- Respond to all review comments. Even if you disagree, explain your reasoning.
- Mark resolved threads as resolved after addressing the comment.
- Do not force-push to a branch after reviews have started without notifying reviewers.
- Ping reviewers in Slack when substantial changes have been made and re-review is needed.

Section 12

API Development Standards

All APIs developed at CoreAxis Technologies follow a contract-first approach. The API contract is defined before implementation begins and serves as the specification for both producers and consumers. CoreAxis primarily uses REST APIs, with GraphQL used in specific data-access-heavy products where agreed with Engineering Leadership.

REST API Conventions

URL Design

- Use nouns for resource names: `/documents`, not `/getDocuments`.
- Use plural nouns for collections: `/embeddings`, not `/embedding`.
- Use kebab-case for multi-word path segments: `/document-chunks`.
- Nest resources to express ownership relationships, to a maximum of two levels: `/collections/{id}/documents`. Deeper nesting is a design smell.
- Query parameters are used for filtering, sorting, and pagination. Path parameters are used for resource identifiers only.

HTTP Methods

Method	Semantics	Idempotent
GET	Read a resource or collection	Yes
POST	Create a new resource	No
PUT	Replace a resource entirely	Yes

PATCH	Partially update a resource	No (by spec)
DELETE	Remove a resource	Yes

HTTP Status Codes

Code	Usage
200	Successful GET, PATCH, PUT
201	Successful POST (resource created); include Location header
204	Successful DELETE (no content to return)
400	Invalid request (validation error)
401	Authentication required
403	Authenticated but not authorised
404	Resource not found
409	Conflict (e.g. duplicate resource)
422	Semantically invalid request
429	Rate limit exceeded
500	Internal server error
503	Service temporarily unavailable

API Versioning

APIs are versioned through the URL path prefix: `/api/v1/`. A new major version is introduced only when breaking changes are required. Non-breaking additions (new fields, new optional parameters, new endpoints) do not require a version bump. Version deprecation must be communicated to API consumers a minimum of 90 days before the old version is decommissioned.

Request and Response Format

- All request and response bodies are JSON (`Content-Type: application/json`).
- Timestamps are ISO 8601 format in UTC: `2025-06-15T09:30:00Z`.
- Monetary values are represented as integers in the lowest denomination (paise, cents) to avoid floating-point precision issues.
- Error responses follow a consistent structure:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "The request body is invalid.",
    "details": [
      { "field": "query", "message": "Field is required." }
    ],
    "request_id": "req_01j9abc123"
  }
}
```

Authentication and Authorisation

All API endpoints require authentication unless explicitly designated as public. Authentication is implemented using JWTs issued by the CoreAxis Identity Service. Bearer token authentication is used: `Authorization: Bearer <token>`. Authorisation is role-based (RBAC) with claims embedded in the JWT payload. All authorisation checks must occur in a dedicated middleware layer — authorisation logic must not be embedded in business logic.

Pagination

Collection endpoints support cursor-based pagination. Page-based pagination is discouraged for large datasets due to consistency issues with concurrent writes. Standard pagination response structure:

```
{
  "data": [...],
  "pagination": {
    "cursor": "eyJpZCI6MTAwfQ==",
    "has_more": true,
    "total": 4821
  }
}
```

Rate Limiting

All public-facing and AI inference endpoints must implement rate limiting. Rate limits are expressed as requests per minute (rpm) and enforced at the API gateway layer. Rate limit headers are returned on every response:

```
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 45
X-RateLimit-Reset: 1718445000
```

Section 13

Database Design Guidelines

Database design is a critical engineering discipline at CoreAxis Technologies. Poor schema design creates performance problems, migration complexity, and data integrity issues that compound over time. These guidelines apply to all relational databases (primarily PostgreSQL) and are adapted for document and vector stores where noted.

PostgreSQL Standards

Naming Conventions

- All table names and column names use `snake_case`.
- Table names are plural nouns: `documents`, `embedding_chunks`, `user_sessions`.
- Boolean columns are prefixed with `is_` or `has_`: `is_active`, `has_been_indexed`.
- Timestamp columns use `_at` suffix: `created_at`, `deleted_at`.
- Foreign key columns reference the table they point to: `user_id` references the `users` table.

Primary Keys

Use `UUID v7` as the default primary key type for new tables. UUID v7 is time-ordered, which preserves index locality while avoiding the sequential enumeration vulnerability of integer PKs in public-facing APIs. For internal join tables with no external exposure, `BIGSERIAL` is acceptable.

Standard Columns

Every table must include:

```
id          UUID DEFAULT gen_random_uuid() PRIMARY KEY,
created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
```

For soft-delete tables, add: `deleted_at TIMESTAMPTZ`

Normalisation

Target third normal form (3NF) by default. Denormalise only when query performance requirements cannot be met at 3NF, and document the trade-off in the technical design document. JSONB columns are acceptable for genuinely schemaless data but must not be used as an escape hatch from proper schema design.

Indexes

- Every foreign key column must have an index.
- Columns used in `WHERE` clauses in frequent queries should have indexes.
- Use `CREATE INDEX CONCURRENTLY` for adding indexes to production tables to avoid locking.
- Analyse index usage quarterly using `pg_stat_user_indexes`. Remove unused indexes to reduce write overhead.
- For full-text search, use PostgreSQL `tsvector`/`tsquery` or delegate to Elasticsearch via the ELK Stack.

Database Migrations

- All schema changes are managed through migration files tracked in version control.
- Migration files are numbered sequentially and never modified after they have been applied to any environment.
- Each migration is atomic — it either completes entirely or rolls back entirely.
- Migrations must be backward-compatible whenever possible. Prefer additive changes (new columns, new tables) over destructive changes (column drops, renames).
- Column drops are executed in two stages: first mark as deprecated (stop writing to it), then drop in a subsequent release after confirming no code references remain.

Redis Usage

Redis is used for caching, session storage, and rate limiting counters. All Redis keys must follow a structured naming convention to avoid collisions across services:

```
<service>:<entity>:<identifier>
Example: rag-service:document:abc123:embeddings
```

All cache entries must have an explicit TTL. Keys without TTL are not permitted.

Vector Database Usage

Vector databases (PostgreSQL with `pgvector`, or dedicated vector stores as agreed with the AI & ML team) store high-dimensional embeddings for similarity search. Key guidelines:

- Store the source document reference alongside each embedding for traceability.
- Use appropriate index types (HNSW, IVFFlat) based on dataset size and recall requirements, as specified by the AI & ML team.
- Embedding dimensions must be consistent within a collection. Mixing embedding models within a collection is not permitted without explicit versioning of the embedding model.

MongoDB Standards

MongoDB is used where document structure is genuinely variable across records. Avoid MongoDB where relational constraints are needed — PostgreSQL is preferred for structured data. In MongoDB:

- Always define an explicit schema validation rule at the collection level.
- Index every field used in queries.
- Use the `ObjectId` as the primary identifier; do not expose it directly in public APIs.
- Enable auditing on collections that store sensitive data.

Section 14

Backend Development Standards

Backend development at CoreAxis Technologies follows Clean Architecture principles applied within the context of our approved backend frameworks: FastAPI and Flask for Python services, and Node.js for JavaScript/TypeScript services. Standards apply across all frameworks unless noted otherwise.

Architecture Layers

Backend services are structured in the following layers, from outermost to innermost:

Layer	Responsibility	Depends On
API / Transport	HTTP routing, request parsing, response serialisation	Application layer
Application	Use case orchestration, transaction boundaries	Domain, Infrastructure interfaces
Domain	Business logic, entities, value objects, domain events	Nothing (no framework imports)
Infrastructure	Database access, external API calls, file I/O, messaging	Domain (implements interfaces)

The Domain layer must not import from any other layer. Infrastructure implements interfaces (Ports) defined in the Domain layer. This inversion is the foundation of testability.

FastAPI / Flask Standards

- Route handlers are thin. Business logic belongs in application-layer services, not in the handler function.
- Request validation uses Pydantic models. No raw `request.json()` or `request.form` access without schema validation.
- All endpoints return responses typed with Pydantic response models. This ensures contract stability and enables automatic OpenAPI documentation.
- Dependency injection uses FastAPI's `Depends` mechanism for injecting services, database sessions, and authentication context.
- Middleware handles cross-cutting concerns: authentication, request logging, CORS, and rate limiting.

Node.js / Express Standards

- Use TypeScript. JavaScript-only Node.js services are not accepted for new projects.
- Request/response schemas are validated using `zod`.
- Route handlers are async functions wrapped in error-catching middleware to prevent unhandled promise rejections.
- Services are injected through constructors or dependency injection containers — not imported as singletons unless they are genuinely stateless utilities.

Background Job Processing

Long-running or resource-intensive operations (document ingestion, embedding generation, report generation) must not execute synchronously within the HTTP request lifecycle. These operations are offloaded to background job queues. Approved queue implementations:

- **Celery + Redis** — for Python services with task queuing requirements.
- **BullMQ** — for Node.js services.

Background job functions must be idempotent. Jobs must be retried on failure up to a configured maximum attempt count. Dead-letter queues are configured for jobs that exhaust their retry budget, and alerts are triggered when dead-letter queue depth exceeds threshold.

AI Microservices and LLM Integration

Services that invoke LLM APIs (OpenAI, Anthropic, self-hosted models via Ollama or Hugging Face) follow these additional standards:

- All LLM API calls are wrapped in a service abstraction that encapsulates retry logic, timeout handling, and token counting.
- Model names and prompt templates are externalised to configuration — not hardcoded in business logic.
- LLM responses are validated before use. Treat LLM output as untrusted external input: do not pass it directly to databases or downstream systems without schema validation.

- Token usage is logged per request for cost tracking. Cost overruns trigger alerts via the monitoring stack.
- Prompt templates are versioned in the configuration store. Changes to prompt templates must go through the same PR review process as code changes.
- AI responses involving sensitive data must be sanitised and audited before surfacing to users. See Section 16 for secure handling of AI outputs.

Model Serving Endpoints

Services that expose custom model inference endpoints (computer vision, custom LLM fine-tunes) must implement:

- Health check endpoints (`/health`, `/ready`) that verify model availability.
- Latency and throughput metrics exported to Prometheus.
- Request validation to prevent prompt injection and oversized payloads.
- Graceful shutdown with in-flight request completion before pod termination.

Section 15

Frontend Development Standards

Frontend development at CoreAxis Technologies is built on React with TypeScript. These standards apply to all web application frontends. Engineers working on frontend code must maintain the same level of rigour in architecture, testing, and code quality as backend engineers.

Project Structure

React applications follow a feature-based directory structure:

```
src/
  components/        # Shared, reusable UI components
```

```
  features/          # Feature modules (each owns its own
components, hooks, services)
  hooks/             # Shared custom hooks
  services/          # API client and service abstractions
  store/             # Global state management (if applicable)
  types/             # Shared TypeScript interfaces and types
  utils/             # Pure utility functions
  pages/             # Route-level components (thin wrappers)
```

State Management

Use the simplest state management solution that satisfies requirements:

- **Component-local state (`useState`):** Default choice for UI state that is not shared.
- **React Context:** Shared state within a feature subtree (theme, authentication context).
- **TanStack Query (React Query):** All server state — fetching, caching, invalidation, and synchronisation.
- **Zustand or Jotai:** Global client state that is complex enough to warrant a dedicated store. Must be justified in the technical design document.

Redux is not approved for new projects due to its ceremony-to-value ratio. Existing Redux codebases should migrate incrementally to TanStack Query + Zustand.

Component Standards

- Components are function components with hooks. Class components are not permitted in new code.
- Components are kept small and focused. A component that exceeds 200 lines is a candidate for decomposition.
- Props are defined as TypeScript interfaces, named `<ComponentName>Props`.
- Avoid prop drilling beyond two levels. Use Context or Zustand for deeply shared state.
- Side effects belong in `useEffect` or custom hooks — not in render logic.
- Memoisation (`React.memo`, `useMemo`, `useCallback`) is used only when profiling confirms a genuine performance issue. Premature memoisation increases cognitive load without benefit.

Accessibility

All UI components must meet WCAG 2.1 Level AA accessibility requirements. Key requirements:

- Interactive elements are keyboard-focusable and operable.
- Colour contrast ratios meet AA minimums (4.5:1 for normal text, 3:1 for large text).
- Images have meaningful `alt` text. Decorative images have `alt=""`.
- Form inputs are associated with labels via `htmlFor` / `id`.
- ARIA attributes are used only where native HTML semantics are insufficient.

Performance

- Lazy-load route-level components using `React.lazy` and `Suspense`.
- Avoid large third-party bundles. Review bundle size with `vite-bundle-analyzer` before merging significant dependency additions.
- Images are served in WebP format with appropriate `srcset` attributes.
- Target a Largest Contentful Paint (LCP) of under 2.5 seconds on a standard mobile network connection.

API Integration

All API calls are made through a typed service layer — not directly from components. TanStack Query hooks encapsulate fetch logic, loading states, and error handling. Axios or the Fetch API with a typed wrapper is used for HTTP requests. API response types are generated from the OpenAPI specification and are not hand-written.

Environment Configuration

Environment-specific configuration is injected through environment variables prefixed with `VITE_` (Vite) or `REACT_APP_` (Create React App). No sensitive values (API keys, secrets) are ever included in frontend environment configuration — these belong only in backend services.

Section 16

Secure Coding Practices

Security is a non-negotiable engineering requirement at CoreAxis Technologies, not a post-development concern. Every engineer is responsible for writing secure code. The Information Security team provides guidance and conducts periodic audits, but the primary responsibility for secure implementation lies with the engineering team.

Input Validation

All data crossing a system boundary — HTTP requests, file uploads, inter-service messages, user-provided prompts — is treated as untrusted and validated at the point of entry. Validation happens server-side. Client-side validation is a UX feature, not a security control.

- Validate type, format, length, and allowable values for every input field.
- Reject inputs that do not conform to the schema. Return a `400 Bad Request` with a descriptive error message.
- For file uploads: validate file type by reading magic bytes, not the client-provided Content-Type header. Enforce maximum file size at the API gateway layer.

SQL Injection Prevention

All database queries use parameterised queries or ORM query builders. String concatenation to build SQL queries is prohibited. Code review will reject any SQL string interpolation from user-supplied inputs.

Cross-Site Scripting (XSS) Prevention

- Never use `dangerouslySetInnerHTML` with user-supplied content. If rich HTML is required, use a sanitisation library (DOMPurify) before rendering.
- Content Security Policy (CSP) headers are configured at the web server / CDN level to restrict executable content sources.
- HTTP response headers are hardened on all services: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Strict-Transport-Security`.

Authentication and Session Management

- JWT tokens have a maximum lifetime of 15 minutes. Refresh tokens are used for session continuity.
- Refresh tokens are stored in secure, HTTP-only cookies — not in local storage.
- All authentication endpoints are rate-limited and protected against credential stuffing.
- Multi-factor authentication (MFA) is required for all internal admin interfaces.

Secret Management

Secrets (API keys, database passwords, signing keys) are never embedded in code, configuration files committed to version control, or logged. See Section 20 for the full secret management policy.

Secure Handling of AI Outputs

LLM outputs present unique security considerations. The following controls are mandatory for all features that process or display LLM-generated content:

- **Prompt injection defence:** User-provided inputs that are incorporated into LLM prompts must be clearly delimited from the system prompt using structural separators. Never concatenate raw user input directly into a prompt.
- **Output sanitisation:** LLM outputs rendered in a web interface are HTML-escaped. LLM outputs incorporated into database writes are schema-validated.
- **Content filtering:** Outputs from external LLM APIs are passed through a moderation check before display in customer-facing interfaces.
- **Sensitive data redaction:** LLM outputs are scanned for PII (names, email addresses, phone numbers, Aadhaar numbers, financial identifiers) before logging or storage. Detected PII is redacted with a placeholder token.

Dependency Security

- All dependencies are scanned for known vulnerabilities using automated tooling in the CI pipeline (Snyk, npm audit, pip-audit).
- High or critical severity vulnerabilities block the build.
- Direct dependencies are pinned to exact versions. Transitive dependencies are managed through lock files (`poetry.lock`, `package-lock.json`, `pnpm-lock.yaml`).

Section 17

Logging Standards

Consistent, structured logging is essential for observability and incident response. Logs are the primary diagnostic tool during production incidents. Engineers must treat logging as a first-class concern — not an afterthought added after debugging becomes difficult.

Log Format

All services emit structured JSON logs. Human-readable text logs are not accepted for production services. Structured logging enables the ELK Stack to index and query log fields efficiently.

Every log entry must include the following fields:

```
{
  "timestamp": "2025-06-15T09:30:00.123Z",
  "level": "info",
  "service": "rag-ingestion-service",
  "version": "1.4.2",
  "environment": "production",
  "trace_id": "01J9ABC123XYZ",
  "message": "Document chunk embedding complete",
  "document_id": "doc_xyz789",
  "chunk_count": 42,
  "duration_ms": 1840
```

```
}
```

Log Levels

Level	When to Use
`DEBUG`	Detailed diagnostic information useful during development. Disabled in production by default.
`INFO`	Significant lifecycle events: service start, request received, operation completed.
`WARN`	Unexpected conditions that do not cause immediate failure but indicate a potential problem.
`ERROR`	A failure that affects a specific request or operation. The system can continue operating.
`FATAL`	A condition that causes the service to shut down. Should trigger an immediate on-call alert.

Approved Logging Libraries

Language	Library
Python	structlog
TypeScript / Node.js	pino
Java	Logback + SLF4J

What to Log

- Service startup and shutdown events (with version and configuration summary).

- All inbound HTTP requests (method, path, status code, duration — no request body by default).
- All outbound calls to external services (target, status, duration).
- All background job start, success, and failure events.
- All LLM API calls (model, token count, latency — no prompt content unless specifically enabled).
- All authentication events (login, logout, token refresh, auth failures).
- All data access events on sensitive resources.

What Not to Log

Never log the following:

- Passwords or password hashes
- JWT tokens or session cookies
- API keys or secret credentials
- Full request bodies containing sensitive fields (PII, financial data)
- LLM prompt content containing user-uploaded documents without explicit audit mode configuration
- Credit card numbers, bank account numbers, Aadhaar numbers

Log Retention

Production logs are retained for 90 days in Elasticsearch. Logs older than 90 days are archived to S3 in compressed format and retained for 12 months to support compliance and audit requirements. Log retention periods are enforced through automated lifecycle policies — do not manually delete logs.

Distributed Tracing

All services propagate trace context through OpenTelemetry. The `trace_id` and `span_id` fields are included in every log entry to enable correlation of logs across service boundaries. The trace_id is also returned in HTTP responses via the `X-Trace-Id` header to support customer-facing error diagnosis.

Section 18

Error Handling Guidelines

Robust error handling is a hallmark of production-ready software. Errors must be caught, classified, logged with context, and surfaced appropriately — to the calling system, to the operator, and in some cases to the end user. Unhandled exceptions that crash services or return cryptic 500 errors to users are engineering failures.

Error Classification

Class	Description	Action
Validation Error	Caller provided invalid input	Return 400/422, log at INFO level, no alert
Authentication Error	Missing or invalid credentials	Return 401, log at INFO level, monitor for spikes
Authorisation Error	Caller lacks permission	Return 403, log at WARN level

Not Found Error	Requested resource does not exist	Return 404, log at DEBUG level
Business Logic Error	Valid input but operation cannot be completed	Return 409/422, log at INFO level
Infrastructure Error	Database, external API, or internal service failure	Return 503/500, log at ERROR level, trigger alert
Unknown Error	Unexpected exception	Return 500, log at ERROR level with full stack trace, trigger alert

Exception Handling in Python

- Define a hierarchy of custom exception classes rooted at a base `CoreAxisError` class.
- Catch exceptions at the appropriate level — not everywhere. Let errors propagate to a centralised exception handler in the framework.
- Never use bare `except:` or `except Exception:` without re-raising or logging with full context.
- Use `structlog.exception()` to log exceptions — this captures the full traceback.

```
class DocumentNotFoundError(CoreAxisError):

    """Raised when a requested document does not exist."""



    def __init__(self, document_id: str) -> None:

        self.document_id = document_id
```

```
        super().__init__(f"Document '{document_id}' not found")
```

Exception Handling in TypeScript

- Define typed error classes that extend the built-in `Error` class.
- Express route handlers are wrapped in an async error-catching utility that forwards unhandled errors to the Express error middleware.
- The global error handler formats errors into the standard error response structure and logs them appropriately.

```
export class ResourceNotFoundError extends Error {

  readonly statusCode = 404;

  readonly code = 'RESOURCE_NOT_FOUND';



  constructor(resource: string, id: string) {

    super(`${resource} with id '${id}' was not found.`);

    this.name = 'ResourceNotFoundError';

  }

}
```

Retries and Circuit Breakers

External service calls (LLM APIs, database connections, third-party REST APIs) are wrapped with retry logic and circuit breakers:

- **Retries:** Transient errors (network timeouts, 429, 503) are retried with exponential backoff and jitter. Maximum of three retry attempts. Non-retryable errors (400, 401, 403, 404) are not retried.
- **Circuit Breaker:** After a configurable number of consecutive failures, the circuit opens and fails fast for a cooldown period before attempting recovery. This prevents cascade failures.

User-Facing Error Messages

Error messages returned to end users must be:

- Informative enough to help the user take corrective action.
- Free of stack traces, internal service names, or system identifiers.
- Accompanied by a `request_id` that support staff can use to locate the server-side log entry.

Section 19

Configuration Management

Configuration management governs how application settings are externalised, validated, and applied across environments. All configuration that varies between environments (development, staging, production) must be externalised from the codebase. Configuration that is constant across all environments may be embedded in code as named constants.

Configuration Sources (Priority Order)

- **Environment variables** — the primary configuration mechanism for all deployed services.
- **Configuration files** — used only for settings that are complex structures unsuitable for environment variables (e.g. Nginx configuration, Kubernetes manifests). Configuration files are committed to the repository without sensitive values.
- **Secrets Manager** — for sensitive values (API keys, database passwords, signing keys). Retrieved at runtime by the service. See Section 20.

Environment Variable Standards

- All environment variable names are `UPPER_SNAKE_CASE`.
- Every service maintains a `.env.example` file in the repository root, listing all required and optional environment variables with descriptions and example (non-sensitive) values.
- Services validate all required environment variables at startup and fail immediately with a clear error message if any required variable is absent or invalid. Services must not start in a degraded or undefined state due to missing configuration.
- Use a configuration module to centralise and validate environment variables — do not scatter `process.env.*` or `os.environ.get()` calls throughout the codebase.

```
# Python example using pydantic BaseSettings

from pydantic_settings import BaseSettings



class Settings(BaseSettings):
```

```
    database_url: str

    redis_url: str

    openai_api_key: str

    log_level: str = "info"

    max_embedding_batch_size: int = 32


    model_config = SettingsConfigDict(env_file=".env")


settings = Settings()
```

Feature Flags

Feature flags are used to control the rollout of AI features, experimental functionality, and significant behavioural changes without requiring a code deployment. Feature flags are implemented through environment variables for simple on/off toggles, or through a feature flag service (LaunchDarkly or equivalent) for percentage rollouts and user-segment targeting.

- Feature flags are named with a `FEATURE_` prefix: `FEATURE_RAG_V2_ENABLED`.
- Flags that have been fully rolled out and are no longer conditional must be removed within two sprints. Stale flags accumulate technical debt.
- The set of active feature flags and their purpose is documented in Confluence.

Configuration Drift

Configuration drift — where environment configurations diverge from their documented state — is a common cause of production incidents. The following practices prevent drift:

- All environment variable changes are deployed through the CI/CD pipeline, not applied manually through cloud console.
- Infrastructure as Code (Terraform, Kubernetes manifests) is the source of truth for all environment configuration.
- Configuration changes are treated as deployments and follow the same change management process as code changes.

Section 20

Environment Management

CoreAxis Technologies operates multiple deployment environments to support the software delivery lifecycle. Each environment serves a distinct purpose and has defined access controls and stability expectations.

Environments

Environment	Purpose	Deployed By	Data Policy
Local Development	Individual engineer workstations	Engineer (manual)	Synthetic or anonymised data only

Development (dev)	Shared integration environment, continuously deployed from main branch	CI/CD (automatic on merge)	Synthetic data only
Staging	Pre-production validation, QA testing, release candidate verification	CI/CD (on release branch cut)	Anonymised production data snapshot
Production	Live customer-facing environment	CI/CD (manual gate)	Live customer data

Secret Management

Secrets are managed through AWS Secrets Manager (primary) or Azure Key Vault (for Azure-deployed services). The following policy governs all secret handling:

- Secrets are never stored in plaintext in version control, configuration files, or Slack/Teams messages.
- Secrets are never logged. Log filtering must be configured to detect and redact accidentally logged secrets.
- Each environment has its own set of secrets. Development secrets must not be used in staging or production.
- Access to production secrets is restricted to production deployment pipelines and designated on-call engineers. Individual engineer access to production secrets requires Security team approval and is logged.
- Secret rotation is performed quarterly for long-lived credentials, or immediately upon suspected compromise.
- Application code retrieves secrets at startup from the Secrets Manager — never at build time.

Environment Access

- Direct shell access to production servers is prohibited except during declared incidents, and is logged and audited.
- All production configuration changes are applied through the CI/CD pipeline, not through manual console edits.
- Engineers receive access to development and staging environments by default. Staging access may be restricted for sensitive projects on approval from the Engineering Manager.
- Access to production systems is granted on a least-privilege basis and reviewed quarterly.

Local Development Environment

Engineers set up their local environments using Docker Compose to replicate dependent services (database, Redis, vector store) locally. A `docker-compose.yml` is maintained in the repository for every service. Local environment setup is documented in the repository README. Engineers must not depend on shared cloud infrastructure for day-to-day local development.

Local development secrets are stored in a `.env` file at the repository root. This file is listed in `.gitignore` and must never be committed. An `.env.example` file with non-sensitive placeholder values is committed instead and kept up to date as the service evolves.

Section 21

Dependency Management

Uncontrolled dependencies are a significant source of security vulnerabilities, build instability, and license compliance risk. CoreAxis Technologies enforces a structured approach to dependency management across all projects.

General Principles

- Use the minimum number of dependencies necessary to meet the requirement. Evaluate whether a dependency is needed before adding it.
- Prefer well-maintained libraries with active communities and clear security disclosure processes.
- Evaluate license compatibility before adding a new dependency. GPL-licensed libraries require review by the Engineering Manager. AGPL-licensed libraries require Legal approval before inclusion.
- All direct dependencies must be pinned to exact versions. Use lock files to ensure deterministic installs across environments.

Python Dependency Management

- Use `poetry` for all Python projects. `requirements.txt` is generated from `poetry.lock` for compatibility but is not the source of truth.
- Separate runtime dependencies from development dependencies in `pyproject.toml`.
- Scan for vulnerabilities using `pip-audit` in the CI pipeline.
- Dependency updates are performed weekly using Dependabot PRs. Security patches are applied within 48 hours of disclosure.

Node.js / TypeScript Dependency Management

- Use `pnpm` as the package manager for all Node.js projects. `npm` and `yarn` are not approved for new projects.
- The `pnpm-lock.yaml` file is committed to version control and kept up to date.
- Scan for vulnerabilities using `pnpm audit` in the CI pipeline.
- Use `npm:audit` thresholds to block builds on high or critical vulnerabilities.

Container Base Image Management

- All Docker images are based on official, minimal base images (Alpine-based or distroless variants preferred).
- Base images are pinned to a specific digest (`sha256:`), not a mutable tag like `latest` or `v3`.
- Images are scanned for vulnerabilities using Trivy in the CI pipeline. High or critical vulnerabilities in base images block the build.
- Base images are updated monthly or upon a critical vulnerability disclosure, whichever is sooner.

AI Model Dependencies

ML model artifacts, embedding model weights, and fine-tuned model checkpoints are treated as versioned dependencies:

- Models are stored in a dedicated model registry (Hugging Face Hub private namespace or AWS S3 versioned bucket), not in the application repository.
- Model versions used in each environment are explicitly pinned in the deployment configuration.
- Model upgrades follow the same PR and review process as code changes, and require validation on the staging environment before promotion to production.

Section 22

Testing Standards

Testing is a core engineering discipline at CoreAxis Technologies. All production code must be tested. The testing strategy follows the testing pyramid: a large base of fast, focused unit tests; a smaller layer of integration tests; and a targeted set of end-to-end tests for critical user journeys.

Test Coverage Requirements

Layer	Minimum Coverage	Measurement
Domain / Business Logic	90%	Line and branch coverage
Application Layer	80%	Line coverage
Infrastructure / Adapters	70%	Line coverage
API Routes	80%	Endpoint coverage (all paths, key status codes)
Frontend Components	70%	Component render + interaction coverage

Coverage below the defined thresholds fails the CI build. Coverage reports are generated and stored as CI artefacts for each pipeline run.

Test Quality Standards

- Tests must be deterministic. Flaky tests must be investigated and fixed within the same sprint they are first observed. A flaky test is as harmful as no test.
- Tests are written in the Arrange-Act-Assert (AAA) pattern.
- Test names clearly describe the scenario: `test_embedding_service_raises_error_when_document_is_empty`.
- Tests do not share mutable state. Each test is fully isolated and can run in any order.
- Tests do not make calls to external services (LLM APIs, production databases). All external dependencies are mocked or stubbed.
- Production-equivalent test data is maintained as fixtures, not generated inline with hardcoded magic values.

Testing for AI Features

AI feature testing presents unique challenges due to non-deterministic model outputs. The following approach is used:

- **LLM output testing:** Use deterministic mock responses in unit and integration tests. Use semantic similarity scoring (not string equality) for evaluating LLM output quality in a dedicated evaluation pipeline.
- **RAG pipeline testing:** Test retrieval precision and recall metrics separately from generation quality. Each stage of the RAG pipeline has its own unit test suite.
- **Regression datasets:** Maintain a curated evaluation dataset of query/expected-output pairs. Run evaluations against this dataset on every release candidate. Track metrics over time to detect degradation.

Test Environments

Unit tests run entirely in-process with no external service dependencies. Integration tests run against locally emulated services (LocalStack for AWS, Docker Compose databases). End-to-end tests run against the staging environment. No test suite runs against the production environment.

Section 23

Unit Testing

Unit tests are the foundation of the testing pyramid. They validate the behaviour of individual functions, classes, and modules in isolation, providing fast feedback during development and preventing regressions.

Approved Frameworks

Language	Framework	Coverage Tool
Python	Pytest	pytest-cov
TypeScript / JavaScript	Vitest	Vitest v8 coverage
Java	JUnit 5 + Mockito	JaCoCo

Python Unit Test Standards

```
import pytest

from unittest.mock import AsyncMock, patch

from app.domain.services.embedding_service import
EmbeddingService
```

```
from app.domain.errors import EmptyDocumentError


class TestEmbeddingService:


    @pytest.fixture
    def service(self, mock_embedding_client):
        return EmbeddingService(client=mock_embedding_client)


    @pytest.fixture
    def mock_embedding_client(self):
        client = AsyncMock()
        client.embed.return_value = [[0.1, 0.2, 0.3]]
        return client


    async def test_embed_returns_vectors_for_valid_document(
        self, service, mock_embedding_client
    ):
        # Arrange
```

```
        document = "CoreAxis is an AI enterprise software
company."


        # Act

        result = await service.embed(document)


        # Assert

        assert result == [[0.1, 0.2, 0.3]]


mock_embedding_client.embed.assert_called_once_with(text=documen
t)


    async def test_embed_raises_error_for_empty_document(self,
service):

        with pytest.raises(EmptyDocumentError):

            await service.embed("")
```

TypeScript Unit Test Standards

```
import { describe, it, expect, vi, beforeEach } from 'vitest';

import { DocumentService } from './document-service';

import { DocumentRepository } from './document-repository';
```

```
describe('DocumentService', () => {
  let service: DocumentService;
  let mockRepo: vi.Mocked<DocumentRepository>;

  beforeEach(() => {
    mockRepo = {
      findById: vi.fn(),
      save: vi.fn(),
    } as unknown as vi.Mocked<DocumentRepository>;
    service = new DocumentService(mockRepo);
  });

  it('should return null when document is not found', async () => {
    // Arrange
    mockRepo.findById.mockResolvedValue(null);

    // Act
```

```
    const result = await service.getDocument('non-existent-id');

    // Assert
    expect(result).toBeNull();

expect(mockRepo.findById).toHaveBeenCalledWith('non-existent-id');
  });
});
```

Mocking Guidelines

- Mock only the direct dependencies of the unit under test — not their dependencies.
- Use interface-based mocking where possible so that mocks remain valid when implementations change.
- Mock external services (LLM APIs, databases, HTTP clients) at the adapter boundary — not deep within business logic.
- Do not over-specify mocks. Asserting that a mock was called with exact arguments is appropriate for important side-effectful calls, but excessive mock assertions make tests brittle.

Pytest Configuration

```
# pyproject.toml

[tool.pytest.ini_options]
```

```
asyncio_mode = "auto"

testpaths = ["tests"]

addopts = "--cov=app --cov-report=term-missing
--cov-fail-under=80"

[tool.coverage.run]

omit = ["tests/*", "app/migrations/*", "*/__init__.py"]
```

Section 24

Integration Testing

Integration tests validate the interaction between components — between the application layer and the database, between services and external APIs, and between different modules within the same service. They provide confidence that the system's parts work together correctly.

Scope

Integration tests in the CoreAxis context cover:

- API endpoint tests: calling real routes with a real application instance and test database.
- Repository tests: exercising database queries against a real (containerised) database instance.

- Service-to-service integration: testing communication between microservices using a local test environment.
- Message queue integration: testing job enqueuing and processing through a local Redis or RabbitMQ instance.

Test Database Strategy

Integration tests run against a dedicated test database instance spun up by Docker Compose. Each test run applies migrations to a clean database. Tests that modify data are wrapped in transactions that are rolled back after each test, keeping the database state clean without the overhead of full teardown and recreation.

FastAPI Integration Testing with Pytest

```
import pytest

from httpx import AsyncClient, ASGITransport

from app.main import create_app

from app.db import get_test_db_session



@pytest.fixture

async def client(test_db_session):

    app = create_app()

    app.dependency_overrides[get_db_session] = lambda:
test_db_session
```

```
    async with AsyncClient(
        transport=ASGITransport(app=app),
        base_url="http://test"
    ) as client:
        yield client


async def test_create_document_returns_201(client,
auth_headers):
    response = await client.post(
        "/api/v1/documents",
        json={"title": "Test Document", "content": "Sample
content"},
        headers=auth_headers
    )
    assert response.status_code == 201
    body = response.json()
    assert body["title"] == "Test Document"
    assert "id" in body
```

Postman / Newman for API Contract Testing

Postman Collections are maintained for every external-facing API. These collections serve as the API contract test suite and are run in the CI pipeline using Newman. The Postman collection tests:

- All documented endpoints respond with the expected status codes.
- Response schemas conform to the OpenAPI specification.
- Authentication and authorisation behave correctly for valid and invalid credentials.
- Rate limiting returns the correct 429 status and headers.

RAG Pipeline Integration Testing

The RAG ingestion and retrieval pipeline has dedicated integration tests that exercise the full data flow:

- Document upload → chunking → embedding → vector store insertion.
- Query → embedding → similarity search → context assembly → response generation.

These tests use a real vector database instance in the test environment and pre-computed embedding fixtures to avoid LLM API calls during testing.

Section 25

End-to-End Testing

End-to-end (E2E) tests validate complete user journeys through the full application stack — from browser interaction through the API to the database and back. They provide confidence that the system works correctly from the user's perspective.

Scope and Strategy

E2E tests are expensive to write and maintain. They are targeted at the most critical and highest-risk user journeys. CoreAxis uses E2E tests for:

- Authentication flows (login, logout, token refresh, MFA).
- Core product workflows (document upload, RAG query, AI response generation).
- Payment and billing flows (where applicable).
- Administrative workflows that, if broken, would prevent customer onboarding.

Non-critical workflows and variations are covered by integration tests, not E2E tests. The total E2E suite must execute within 15 minutes to avoid becoming a bottleneck in the CI pipeline.

Approved Framework: Playwright

Playwright is the approved E2E testing framework for all web applications. Selenium is maintained for legacy projects but must not be used for new projects.

```
import { test, expect } from '@playwright/test';



test('user can submit a RAG query and receive a response', async
({ page }) => {

  // Arrange

  await page.goto('/login');

  await page.fill('[data-testid="email-input"]',
'test.user@coreaxis.ai');
```

```
  await page.fill('[data-testid="password-input"]',
process.env.TEST_USER_PASSWORD!);

  await page.click('[data-testid="login-button"]');

  await expect(page).toHaveURL('/dashboard');


  // Act

  await page.click('[data-testid="new-query-button"]');

  await page.fill('[data-testid="query-input"]', 'What is the
refund policy?');

  await page.click('[data-testid="submit-query-button"]');


  // Assert

  const response =
page.locator('[data-testid="query-response"]');

  await expect(response).toBeVisible({ timeout: 30_000 });

  await expect(response).not.toBeEmpty();

});
```

Test Data Management

E2E tests run against the staging environment. Test data is provisioned through an API-level setup script that runs before the test suite and cleaned up afterwards. E2E

tests must not rely on data that could be modified by other tests running in parallel. Test accounts and datasets are isolated per test run using unique identifiers.

Visual Regression Testing

For products with stable, well-defined UI components, visual regression screenshots are captured using Playwright's built-in screenshot comparison. Visual regression tests run weekly and on release branches. Failures are reviewed by the frontend team before a release is promoted to production.

E2E Test Maintenance

E2E tests are brittle relative to unit and integration tests. To manage this:

- Use `data-testid` attributes for element selectors — never rely on CSS class names or DOM structure.
- Use Playwright's built-in retry and wait mechanisms instead of fixed-time `sleep` calls.
- A failing E2E test in the CI pipeline that cannot be fixed within 24 hours is temporarily quarantined (skipped with a Jira ticket linked) to unblock the pipeline. Quarantine is not a permanent state — tickets have a two-sprint SLA for resolution.

Section 26

CI/CD Pipeline Standards

All software delivery at CoreAxis Technologies is automated through CI/CD pipelines implemented in GitHub Actions. No code reaches any shared environment through manual intervention. The pipeline is the only path from source code to deployment.

Pipeline Stages

Stage	Trigger	Actions	Must Pass?
Validate	Every push	Lint, type check, format check	Yes
Test	Every push	Unit tests + coverage report	Yes
Security Scan	Every push	Dependency audit, SAST scan, secret detection	Yes (blocks on high/critical)
Build	PR to main, merge to main	Compile, build Docker image, push to registry	Yes
Integration Test	Merge to main	API contract tests, service integration tests	Yes
Deploy Dev	Merge to main	Apply manifests to dev cluster	Yes
Deploy Staging	Release branch cut	Apply manifests to staging cluster	Yes
E2E Test	After staging deploy	Playwright E2E suite against staging	Yes

Deploy Production	Manual approval gate	Apply manifests to production cluster	Approval required

GitHub Actions Conventions

- Workflow files are stored in `.github/workflows/`.
- Reusable steps are extracted as composite actions in `.github/actions/`.
- Actions are pinned to a specific commit SHA, not a mutable tag: `actions/checkout@a81bbbf`.
- Secrets are injected from GitHub Secrets — never hardcoded in workflow files.
- Workflow runs on ephemeral GitHub-hosted runners or CoreAxis self-hosted runners for jobs requiring internal network access.

Deployment Approval Gate

Production deployments require a manual approval from at least one of: Engineering Manager, Technical Lead, or designated Release Manager. The approval is recorded in the GitHub Actions audit log. Emergency hotfix deployments may be approved by a single on-call Technical Lead.

Build Artefacts

- Docker images are tagged with the Git commit SHA and the semantic version (when a release is cut).
- Images are pushed to the CoreAxis private registry on AWS ECR.
- The `latest` tag is not used in production — explicit versioned tags are required.

Rollback

Every deployment is reversible. The rollback procedure is:

- Identify the last known good image tag from the deployment history.
- Update the Kubernetes deployment manifest to reference the previous image tag.
- Apply the rollback through the CI/CD pipeline using the rollback workflow, not manually with `kubectl`.
- Notify stakeholders of the rollback and file an incident report.

The CI/CD pipeline includes a dedicated rollback workflow (`rollback.yml`) that accepts an image tag as input and deploys it without requiring the full pipeline to execute.

Section 27

Docker Container Standards

All services at CoreAxis Technologies are containerised using Docker. Containers provide environment consistency, deployment portability, and process isolation. Engineers are responsible for maintaining production-quality Dockerfiles for their services.

Dockerfile Requirements

- Use multi-stage builds to separate the build environment from the runtime image. The final image contains only runtime artefacts — no build tooling, compilers, or source code.
- Base images are minimal: prefer Alpine or distroless variants. The `ubuntu` or `debian` full images are not used unless a specific library requires it.

- Base images are pinned to a digest: `FROM python:3.11-alpine@sha256:abc123`.
- The application runs as a non-root user. Create a dedicated application user in the Dockerfile and switch to it with `USER app`.
- Sensitive build arguments are not embedded in the image. Pass them at build time and ensure they do not appear in image layers.

Example: Python FastAPI Service Dockerfile

```
# Stage 1: Build

FROM python:3.11-alpine AS builder

WORKDIR /build

RUN pip install poetry

COPY pyproject.toml poetry.lock ./

RUN poetry export --without-hashes -f requirements.txt -o
requirements.txt



# Stage 2: Runtime

FROM python:3.11-alpine AS runtime

WORKDIR /app

RUN addgroup -S app && adduser -S app -G app

COPY --from=builder /build/requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt

COPY --chown=app:app src/ ./src/

USER app

EXPOSE 8000

CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Image Size and Layers

- Combine related `RUN` commands into a single layer using `&&` to minimise layer count.
- Clean up package manager caches in the same `RUN` instruction that installs packages: `apt-get clean && rm -rf /var/lib/apt/lists/*`.
- Production images are scanned for size regression in CI. Images that grow more than 20% without justification are flagged for review.

Container Security

- Images are scanned with Trivy in CI. High and critical vulnerabilities block the build.
- Containers run with a read-only root filesystem where possible. Writable paths (log directories, temporary storage) are explicitly mounted as volumes.
- No privileged containers in production. Capabilities are dropped to the minimum required by the service.
- Container images are signed with Docker Content Trust (DCT) before pushing to the registry.

Docker Compose for Local Development

Each service repository includes a `docker-compose.yml` for local development. The Compose file defines all service dependencies (PostgreSQL, Redis, the service itself) with appropriate environment variable defaults and health checks. Engineers must be able to run the full service stack locally with a single `docker compose up` command.

Section 28

Kubernetes Deployment Standards

All production and staging services at CoreAxis Technologies run on Kubernetes. Kubernetes deployment configuration is managed as code in a dedicated infrastructure repository and applied through the CI/CD pipeline. Manual `kubectl apply` commands on production clusters are prohibited outside of declared incidents.

Namespace Organisation

Services are organised into namespaces by environment and product area:

```
coreaxis-dev        # Development environment

coreaxis-staging    # Staging environment

coreaxis-prod       # Production environment
```

Within each environment namespace, Kubernetes labels are used to distinguish services by product domain.

Required Resources per Service

Every deployed service must have the following Kubernetes resources defined:

- **Deployment:** Defines the Pod template, replica count, image tag, and update strategy.
- **Service:** Exposes the Deployment within the cluster.
- **Ingress (or Gateway):** Routes external traffic to the service via Nginx Ingress Controller.
- **ConfigMap:** Non-sensitive environment configuration.
- **HorizontalPodAutoscaler (HPA):** Automatic scaling based on CPU and memory utilisation.
- **PodDisruptionBudget (PDB):** Ensures minimum availability during node maintenance.
- **ResourceQuota:** CPU and memory requests and limits on every container.

Resource Requests and Limits

Policy: Every container must have CPU and memory resource requests and limits defined. Containers without resource limits are not permitted in staging or production. Pods without resource requests cannot be reliably scheduled.

```
resources:

  requests:

    cpu: "250m"

    memory: "512Mi"

  limits:

    cpu: "1000m"
```

```
    memory: "1Gi"
```

Health Checks

All services must define liveness and readiness probes:

- **Readiness probe:** Checked before routing traffic to a pod. Must verify that the application is fully initialised, database connections are established, and the service is ready to handle requests.
- **Liveness probe:** Checked periodically during operation. Signals that the process is running correctly. Failures trigger a pod restart.

```
livenessProbe:
  httpGet:
    path: /health/live
    port: 8000
  initialDelaySeconds: 15
  periodSeconds: 20
  failureThreshold: 3


readinessProbe:
  httpGet:
    path: /health/ready
    port: 8000
```

```
  initialDelaySeconds: 10

  periodSeconds: 10

  failureThreshold: 3
```

Deployment Strategy

Production deployments use a rolling update strategy with the following parameters:

```
strategy:

  type: RollingUpdate

  rollingUpdate:

    maxUnavailable: 0

    maxSurge: 1
```

Zero-downtime deployments are mandatory for all production services. The `maxUnavailable: 0` setting ensures at least the current replica count is always available during a rollout.

AI Model Workload Considerations

- GPU workloads (model inference, embedding generation) are deployed to GPU-enabled node pools with appropriate node selectors and tolerations.
- Model serving pods have a startup probe in addition to liveness and readiness probes to account for model loading time.
- GPU resource limits are set explicitly: `nvidia.com/gpu: 1`.

Section 29

Release Management

Release management governs how software versions are defined, communicated, and deployed to production. A structured release process reduces deployment risk, ensures stakeholder visibility, and provides a clear audit trail of what was deployed and when.

Semantic Versioning

All CoreAxis services and libraries use Semantic Versioning (SemVer): `MAJOR.MINOR.PATCH`.

Version Component	When to Increment
MAJOR	Breaking changes to public API contracts; incompatible data schema changes
MINOR	New backwards-compatible features; new optional API parameters
PATCH	Backwards-compatible bug fixes; security patches

Release Process

1. **Release Planning:** The Release Manager (Engineering Manager or Technical Lead) identifies the stories included in the release, reviews open defects, and confirms the release scope two sprints before the release date.

- **Release Branch Cut:** A release branch is created from `main` at the sprint end. No new features are merged to the release branch after cut. Only defect fixes and security patches are cherry-picked.
- **Staging Validation:** The release candidate is deployed to staging. QA Engineers conduct acceptance testing. Critical defects found in staging must be fixed and the release candidate rebuilt before promoting to production.
- **Release Notes:** The Release Manager produces release notes in Confluence listing: new features, bug fixes, known issues, breaking changes, and migration instructions if applicable.
- **Production Deployment:** Deployment is executed through the CI/CD pipeline with the required approvals. Deployment is scheduled during a low-traffic maintenance window where possible.
- **Post-Deployment Verification:** Within 30 minutes of deployment, the release team verifies core functionality in production using a production smoke test suite. Dashboards are monitored for anomalies for the first two hours post-deployment.

Git Tagging

Every production release is tagged in Git with the version number and a signed tag:

```
git tag -s v2.3.1 -m "Release v2.3.1: RAG document chunking improvements"
```

```
git push origin v2.3.1
```

Release tags are permanent. They may not be force-deleted or moved after creation.

Rollback Criteria

A rollback is initiated when any of the following conditions are observed within two hours of a production deployment:

- Error rate exceeds the pre-deployment baseline by more than 5%.
- P95 response latency increases by more than 25%.
- A critical severity incident is raised that is causally linked to the deployment.
- A core user journey fails in production smoke tests.

The decision to rollback rests with the on-call Technical Lead or Engineering Manager. Rollback is preferred over attempting to forward-fix an unknown production issue under time pressure.

Section 30

Monitoring and Observability

Observability is the capacity to understand the internal state of a system from its external outputs. At CoreAxis Technologies, observability is built on three pillars: metrics, logs, and traces. All production services are required to emit all three.

Metrics — Prometheus + Grafana

All services expose a `/metrics` endpoint in Prometheus exposition format. Prometheus scrapes this endpoint on a configurable interval. Grafana provides dashboards for visualisation and alerting.

Every service must expose the following baseline metrics:

- **Request rate:** HTTP requests per second, labelled by method, path, and status code.
- **Error rate:** Percentage of requests returning 5xx status codes.
- **Request latency:** P50, P90, P95, P99 latency histograms, labelled by endpoint.

- **Saturation:** CPU utilisation, memory utilisation, database connection pool usage.

AI-specific services additionally expose:

- LLM token consumption rate (input and output tokens per minute).
- Embedding generation throughput (chunks per second).
- RAG query latency (time to retrieve relevant context and generate response).
- Model inference queue depth and processing latency.

Alerting Policy

Alert Severity	Condition	Response SLA
Critical	Service unavailable; error rate above 10%; SLA breach imminent	15 minutes
High	Error rate elevated; latency degraded; capacity approaching limit	1 hour
Warning	Trend indicating possible future degradation; non-critical errors	Next business day

Logs — ELK Stack

All service logs are shipped to the ELK Stack (Elasticsearch, Logstash, Kibana) via a Filebeat sidecar container. Structured JSON logs (see Section 17) are indexed by Elasticsearch and queried through Kibana. Dashboards for each service track error rates, slow queries, and anomalous log patterns.

Traces — OpenTelemetry

Distributed traces are collected using OpenTelemetry SDKs instrumented in each service. Traces are exported to Jaeger or AWS X-Ray depending on the deployment environment. Each service automatically creates spans for:

- Inbound HTTP request handling.
- Outbound HTTP calls to downstream services and external APIs.
- Database query execution.
- LLM API calls (including model name and token count as span attributes).
- Background job processing.

Service Level Objectives (SLOs)

Each production service has defined SLOs published in Confluence. Standard SLOs are:

- **Availability:** 99.9% over a rolling 30-day window.
- **API latency P95:** Under 500ms for synchronous endpoints.
- **AI inference latency P90:** Under 3 seconds for RAG query responses (configurable per product SLA).

SLO compliance is reviewed monthly by Engineering Leadership. Breaches trigger an obligatory post-incident review and reliability improvement sprint stories.

Section 31

Performance Optimisation

Performance at CoreAxis Technologies is treated as a feature, not an afterthought. Engineers are expected to write performant code from the outset. When performance

issues arise in production, they are investigated systematically using profiling and observability data — not remedied through guesswork.

Performance Budgets

Before beginning a feature, identify the performance budget — the maximum acceptable latency, throughput, and resource consumption for the operation. Performance budgets are defined in the technical design document and validated against during load testing before production deployment.

Backend Performance

Database Query Optimisation

- All database queries executed in production are analysed with `EXPLAIN ANALYZE` before deployment. Sequential scans on large tables must be justified.
- Use index-only scans where possible by designing indexes to cover query patterns.
- Batch database operations: insert and update multiple rows in a single query rather than in a loop.
- Use connection pooling (PgBouncer for PostgreSQL) to avoid the overhead of establishing new connections per request.
- N+1 query problems are caught during code review. Use eager loading or query batching (DataLoader pattern) for related data access.

Caching Strategy

- Cache the results of expensive operations at an appropriate granularity using Redis.
- Cache invalidation is explicit — not TTL-only for data that must be consistent. Invalidate on write.

- Cache AI inference results for deterministic or near-deterministic queries. Cache key design must account for all variables that affect the output (model version, prompt template version, input hash).
- Cache hit rate is monitored in Grafana. A hit rate below 70% on a cache that is expected to be effective triggers a cache strategy review.

Async and Concurrent Processing

- I/O-bound operations (external API calls, database queries) are executed asynchronously using async/await.
- Independent operations within a request are executed concurrently using `asyncio.gather` (Python) or `Promise.all` (TypeScript).
- CPU-bound tasks are offloaded to worker processes or background job queues to avoid blocking the event loop.

Frontend Performance

- Lighthouse scores are measured in CI for every PR that touches frontend code. Target scores: Performance ≥ 85, Accessibility ≥ 95, Best Practices ≥ 90.
- Largest Contentful Paint (LCP) must not regress by more than 200ms compared to the main branch baseline.
- JavaScript bundle size is tracked. Bundle additions that increase the total by more than 10% require Engineering Manager review.

AI Workload Performance

- Embedding generation is batched. Send multiple document chunks in a single API call rather than one call per chunk.

- Use streaming responses for LLM outputs to reduce perceived latency for the end user — the response begins displaying as soon as the first tokens are generated.
- Model quantisation (INT8, INT4) is evaluated for on-premise deployments where latency is acceptable at reduced precision.
- Concurrent LLM API requests are rate-limited at the application level to prevent exceeding quota and incurring rate limit errors.

Section 32

Documentation Standards

Documentation is a first-class engineering output at CoreAxis Technologies. Code that is not documented cannot be maintained reliably by anyone other than its original author. Engineers are expected to produce and maintain documentation as an integral part of their work, not as a post-completion task.

Documentation Levels

Level	What	Where	Audience
Code-level	Docstrings, JSDoc, inline comments	Source code repository	Engineers working on the codebase
API-level	OpenAPI specification, Postman collection	Repository + Postman workspace	API consumers (internal and external)

Component-level	README, Architecture Decision Records	Repository	Engineers onboarding to the service
System-level	System design documents, data flow diagrams	Confluence	Technical audience across teams
Operational	Runbooks, incident response playbooks	Confluence	On-call engineers, DevOps
User-facing	Product documentation, API guides	External docs portal	Customers, partner developers

Repository README Requirements

Every service repository must have a `README.md` containing:

- Service name and one-sentence purpose statement.
- System context: what this service does and how it fits into the broader architecture.
- Prerequisites for local development.
- Step-by-step local setup instructions (including Docker Compose).
- Complete environment variable reference with descriptions and default/example values.
- Instructions for running tests.
- Link to the API documentation (OpenAPI spec or Confluence).
- On-call contact and Confluence runbook link.

Architecture Decision Records (ADRs)

Significant technical decisions are documented as Architecture Decision Records (ADRs) stored in a `docs/adr/` directory in the repository. ADRs follow this template:

```
# ADR-0012: Use pgvector for Embedding Storage


## Status

Accepted


## Context

The RAG pipeline requires vector similarity search over document embeddings.

Options evaluated: pgvector, Pinecone, Qdrant, Weaviate.


## Decision

Adopt pgvector as the vector store for the initial production deployment.


## Rationale

Reduces operational complexity by consolidating storage in the existing
```

```
PostgreSQL instance. Acceptable recall performance (>95%) at our current

dataset scale (<10M vectors). Cost-effective vs. managed vector database services.


## Consequences

- Positive: Single database to manage; existing operational expertise.

- Negative: May require migration to a dedicated vector store at 50M+ vector scale.

- Review trigger: Monthly evaluation of query latency P95 against SLO threshold.
```

Confluence Documentation Standards

- System design documents use the standard CoreAxis Design Document template in Confluence.
- Documents are reviewed and updated at each major release. Stale documents (not updated in more than six months for active systems) are flagged for review.
- Runbooks are tested quarterly — engineers follow the documented steps against a non-production environment and update the runbook if discrepancies are found.

Section 33

Collaboration with AI & Machine Learning Team

The Software Engineering and AI & Machine Learning (AI & ML) teams at CoreAxis Technologies work in close collaboration to integrate AI capabilities into products. This section defines the operating model for that collaboration.

Model Handoff Protocol

When the AI & ML team delivers a model or model update for integration into a product, the handoff package must include:

- Model artefact stored in the agreed model registry with a versioned identifier.
- Inference API specification (input format, output format, expected latency, token limits).
- Evaluation report: benchmark metrics, known failure modes, and recommended use cases.
- Integration example code (Python or TypeScript) demonstrating correct invocation.
- Monitoring guidance: which output characteristics should be monitored in production.

RAG Pipeline Collaboration

RAG pipeline development involves both teams. Responsibilities are divided as follows:

Responsibility	Owner
Embedding model selection and evaluation	AI & ML
Chunking strategy and parameters	AI & ML (with SE input on data scale)
Vector store integration and infrastructure	Software Engineering
Retrieval pipeline implementation	Software Engineering
Prompt template design and versioning	AI & ML (reviewed by SE for security)
LLM API integration and retry logic	Software Engineering
End-to-end RAG quality evaluation	AI & ML
Production monitoring and alerting	Software Engineering (DevOps)

AI API Gateway

All LLM and embedding API calls from production services are routed through the CoreAxis AI API Gateway. The gateway provides:

- Centralised authentication and quota management across LLM providers.
- Request and response logging for cost tracking and debugging.
- Failover between LLM providers when primary provider is unavailable.
- Rate limiting per service and per user to prevent runaway costs.
- Response caching for repeated identical queries.

Direct calls to LLM provider APIs that bypass the gateway are not permitted in production code.

Prompt Management

Prompt templates used in production are managed as versioned artefacts in the prompt registry (maintained by AI & ML). Software Engineering services reference prompts by ID and version:

```
# Python example

from coreaxis.prompts import PromptRegistry



prompt = PromptRegistry.get("rag_query_v3")

rendered = prompt.render(context=retrieved_chunks,
query=user_query)
```

Changes to prompt templates follow a review process that includes both AI & ML and Software Engineering. Prompt changes that affect API response format must be treated as breaking changes and versioned accordingly.

Section 34

Collaboration with IT Department

The IT Department at CoreAxis Technologies provides the infrastructure, tooling, and support that the engineering team depends on. Clear collaboration boundaries and escalation paths ensure that IT can support engineering needs without creating bottlenecks or ambiguity.

IT-Managed Resources

The following resources are owned and managed by the IT Department:

- Corporate network infrastructure (VPN, DNS, corporate Wi-Fi).
- Engineer workstations, hardware procurement, and asset management.
- Corporate identity management (Active Directory, SSO).
- Office productivity tooling (email, calendaring, collaboration tools).
- Corporate network security monitoring.

Engineering-Managed Resources

The following resources are owned and managed by the Software Engineering Department:

- Cloud infrastructure on AWS and Azure (provisioned through Terraform, owned by DevOps).
- GitHub Enterprise organisation and repository access.
- CI/CD pipelines and build infrastructure.
- Application-level monitoring (Prometheus, Grafana, ELK Stack).
- Container registries and artefact repositories.

Access Requests

Requests for new system access or tool procurement must be submitted through the IT helpdesk ticketing system. Engineering Managers are responsible for approving access requests from their direct reports. Access provisioning SLA is two business days for standard requests. Urgent access requests (on-call emergency, new starter with imminent start date) are escalated through the Engineering Manager to IT leadership.

Network and Security Coordination

Engineering changes that affect network topology, open new inbound ports, or introduce new data flows to external services must be communicated to IT at least five business

days in advance. This allows IT to update firewall rules, network monitoring, and access control lists before the change is deployed.

Workstation Standards

Engineer workstations must comply with IT-mandated security policies:

- Full-disk encryption is enabled.
- The IT-approved endpoint detection and response (EDR) agent is installed and running.
- Operating system security updates are applied within 14 days of release.
- Screen lock is configured to activate after 5 minutes of inactivity.

Engineers who require software not available through the approved software catalogue must submit a procurement request. IT processes procurement requests within five business days. Engineers must not install unapproved software that requires elevated system privileges.

Section 35

Collaboration with Information Security

The Information Security (InfoSec) team at CoreAxis Technologies is responsible for defining security policy, conducting security assessments, and supporting engineering teams in building secure software. Engineering and InfoSec collaboration is continuous — not a checkpoint at the end of a release.

Security Review Process

The following engineering activities require a mandatory security review before proceeding to production:

- New customer-facing APIs or changes to authentication and authorisation logic.
- New data integrations that receive or transmit sensitive or personal data.
- Changes to encryption, hashing, or key management implementations.
- Introduction of new third-party services that receive CoreAxis customer data.
- Significant new AI features that process user-uploaded content or generate responses containing customer data.

Security review requests are submitted through Jira under the InfoSec project, minimum five business days before the target production deployment date.

Vulnerability Management

- Critical vulnerabilities (CVSS ≥ 9.0) must be remediated within 24 hours of disclosure.
- High vulnerabilities (CVSS 7.0–8.9) must be remediated within 7 days.
- Medium vulnerabilities (CVSS 4.0–6.9) must be remediated within 30 days.
- Remediation timelines that cannot be met must be escalated to InfoSec with a documented compensating control.

Penetration Testing

InfoSec conducts penetration tests against production systems annually and on significant architectural changes. Engineering teams are responsible for providing test accounts, API documentation, and a test environment for the penetration testing exercise. Findings from penetration tests are triaged and prioritised jointly by InfoSec and the relevant Engineering Manager.

Security Champion Programme

Each engineering team designates a Security Champion — an engineer who acts as the InfoSec team's point of contact within the team. Security Champions:

- Attend the monthly Security Champion sync with InfoSec.
- Review security-related items in sprint planning and raise concerns early.
- Conduct lightweight threat modelling on new features before implementation begins.
- Ensure the team's dependency scanning and SAST results are triaged promptly.

Data Classification and Handling

Data Class	Examples	Handling Requirements
Public	Marketing content, public API docs	No restrictions
Internal	Engineering designs, internal tooling	Not shared externally without approval
Confidential	Customer data, financial records, employee data	Encrypted at rest and in transit; access controls enforced
Restricted	PII, health data, authentication credentials	Encrypted + access logged + InfoSec approval for new integrations

Section 36

Technical Debt Management

Technical debt is the accumulated cost of shortcuts, deferred improvements, and outdated decisions in a codebase. Left unmanaged, technical debt slows development, increases defect rates, and makes onboarding harder. CoreAxis Technologies treats technical debt as a managed liability — not an inevitable fact of engineering life.

Technical Debt Register

Technical debt items are logged as Jira stories under the label `tech-debt`. Each entry includes:

- Description of the debt and why it exists.
- Impact: what is harder or more risky because of this debt.
- Estimated effort to resolve.
- Priority: High / Medium / Low based on impact on development velocity and system risk.

The technical debt register is reviewed by the Technical Lead and Engineering Manager at each sprint retrospective. High-priority items are scheduled into the next available sprint. Medium-priority items are addressed within three sprints. Low-priority items are addressed opportunistically.

Budget for Technical Debt

Each sprint allocates a minimum of 20% of story point capacity to technical debt reduction. This allocation is sacrosanct — it is not re-allocated to feature work even

when delivery pressure is high. Engineering Managers enforce this allocation and report compliance to Engineering Leadership.

Preventing New Technical Debt

The following practices prevent new debt from being created:

- **No shortcuts under time pressure without documentation.** If a shortcut is taken because of a deadline, create a tech-debt Jira ticket before merging the PR and link it in the PR description. Undocumented shortcuts are not acceptable.
- **Refactoring as part of feature work.** Engineers follow the Boy Scout Rule: leave the code in a better state than you found it. Minor refactoring within the scope of a feature change is encouraged and does not require a separate ticket.
- **Architecture reviews for significant new work.** New features that touch core infrastructure or introduce significant new complexity require an architecture review to prevent creating structural debt.

Legacy System Migration

Legacy systems identified for modernisation are migrated incrementally using the Strangler Fig pattern — new functionality is built in the new system while the legacy system is gradually replaced rather than rewritten wholesale. Migration projects require a documented migration plan approved by Engineering Leadership before beginning.

Section 37

Incident Response During Production

Production incidents are events that negatively affect the availability, performance, or correctness of CoreAxis software in ways that impact customers or internal operations. A structured incident response process minimises customer impact and accelerates resolution.

Incident Severity Levels

Severity	Definition	Initial Response SLA
P1 — Critical	Complete service outage; all customers impacted; data loss or security breach	15 minutes
P2 — High	Major functionality unavailable; significant customer impact; SLA breach	30 minutes
P3 — Medium	Degraded performance; partial functionality impacted; limited customer impact	2 hours
P4 — Low	Minor issue; single customer affected; no SLA impact	Next business day

Incident Response Roles

- **Incident Commander (IC):** Coordinates the response. Makes the call to escalate, rollback, or invoke a war room. Usually the on-call Engineering Manager or Technical Lead.
- **Technical Lead:** Investigates root cause, coordinates engineers on diagnosis and remediation.
- **Communications Lead:** Maintains the incident Slack channel, posts status updates, coordinates with Customer Success.

- **Scribe:** Maintains a real-time chronological log of events, decisions, and actions taken during the incident.

Incident Response Procedure

- **Detect and Triage:** The on-call engineer receives an alert or customer report, assesses severity, and declares an incident in the `#incidents` Slack channel.
- **Assemble Response Team:** The IC assembles the appropriate engineers via PagerDuty escalation.
- **Stabilise:** Prioritise customer impact reduction. This may mean rolling back, disabling a feature flag, or adding capacity — before the root cause is known.
- **Diagnose:** Investigate root cause in parallel with stabilisation. Use logs, traces, and metrics to narrow down the cause. Do not speculate.
- **Remediate:** Apply the fix (code change, configuration update, or infrastructure action) through the standard deployment pipeline where possible.
- **Verify:** Confirm resolution using monitoring dashboards and, for P1/P2, customer-side verification.
- **Close:** Declare the incident resolved in `#incidents`. Assign a post-mortem owner.

Post-Incident Review

All P1 and P2 incidents, and P3 incidents involving customer data, require a Post-Incident Review (PIR) completed within five business days of resolution. The PIR follows a blameless culture — the goal is to understand contributing factors and improve the system, not to assign blame to individuals. The PIR document is stored in Confluence and includes:

- Incident summary and customer impact.

- Chronological timeline of events.
- Root cause analysis (5 Whys or equivalent).
- Contributing factors.
- Action items with assigned owners and due dates.

Section 38

Employee Responsibilities

Engineers at CoreAxis Technologies operate as trusted professionals with access to sensitive systems, customer data, and internal intellectual property. With this trust comes defined responsibilities that each engineer is expected to uphold consistently.

Code of Conduct

- Treat all colleagues with respect. Disagreements on technical matters are resolved through constructive discussion, evidence, and where necessary, escalation to the Technical Lead or Engineering Manager.
- Code reviews are not personal critiques. Review comments focus on the code, not the author. Dismissive, condescending, or aggressive review language is not acceptable and will be addressed through HR processes.
- Credit for work is attributed accurately. Do not claim sole authorship of collaborative work. Acknowledge contributions from colleagues in documentation and communications.

Intellectual Property

- All code written during employment at CoreAxis Technologies belongs to CoreAxis Technologies.

- Do not incorporate open-source code with incompatible licences into proprietary codebases without Legal review.
- Do not share proprietary code, algorithms, or architectural designs externally without Engineering Leadership approval.
- Do not use AI coding assistants to generate code that is then submitted to a proprietary codebase without reviewing the output for licence-incompatible training data implications. Consult Engineering Leadership if uncertain.

Data Handling

- Access only the data required to perform your assigned work. Do not query or export customer data for curiosity or convenience beyond what is operationally necessary.
- Production data must never be copied to local development machines or personal storage. Use anonymised test datasets for development purposes.
- Report any suspected data breach or unauthorised access immediately to InfoSec and your Engineering Manager, regardless of whether you believe the exposure was limited.

Security Responsibilities

- Complete all mandatory security awareness training within the deadlines set by InfoSec.
- Report phishing attempts and suspicious emails to IT security without clicking any links.
- Do not use personal devices to access production systems or customer data unless explicitly permitted under the BYOD policy.
- Lock workstations when stepping away, even briefly.

Availability and Communication

- Respond to Slack messages during core hours within a reasonable timeframe. Engineers are not expected to be continuously available but should not be unreachable for extended periods without notice.
- Planned absences (leave, medical appointments, off-site work) are communicated to the Engineering Manager and team in advance.
- On-call rotations are honoured. Engineers on call must be reachable and capable of responding within the defined SLA window. On-call schedule conflicts must be communicated to the Engineering Manager at least one week in advance.

Section 39

Continuous Learning

The technology landscape, particularly in AI and cloud-native infrastructure, evolves rapidly. CoreAxis Technologies is committed to supporting engineers in staying current with relevant technologies, industry practices, and professional skills. Continuous learning is both a personal responsibility and an organisational investment.

Learning Budget

Each engineer is allocated an annual learning budget of ₹25,000 for professional development. This budget may be used for:

- Online courses and certifications (Coursera, Udemy, A Cloud Guru, Pluralsight).
- Technical conference registrations and travel (approval from Engineering Manager required for in-person events).
- Technical books and subscriptions.

- Certification exam fees (AWS, Azure, Google Cloud, Kubernetes, etc.).

Learning budget requests are submitted through the HR portal and approved by the Engineering Manager. Unused budget does not carry over to the following year.

Internal Learning

- **Engineering Guild Meetings:** Bi-weekly cross-team sessions where engineers share learnings, demonstrate new tools, or present solutions to interesting problems. Attendance is strongly encouraged.
- **Tech Radar:** Engineering Leadership maintains a quarterly Tech Radar document in Confluence, categorising technologies as Adopt, Trial, Assess, or Hold. Engineers are encouraged to contribute assessments of new tools they encounter.
- **Pair Programming:** Engineers are encouraged to pair on complex problems or when working in unfamiliar areas of the codebase. Pairing accelerates knowledge transfer and reduces knowledge silos.
- **Architecture Brown Bags:** Monthly sessions hosted by Technical Leads covering significant architectural decisions, system designs, and post-incident learnings.

Certifications

Relevant certifications are recognised and supported by CoreAxis Technologies. Encouraged certifications include:

- AWS Solutions Architect Associate or Professional.
- Google Cloud Professional Data Engineer or Machine Learning Engineer.
- Certified Kubernetes Application Developer (CKAD) or Administrator (CKA).
- HashiCorp Terraform Associate.

Engineers who complete an approved certification receive recognition in the engineering all-hands and may be asked to share learnings at a Guild meeting.

Performance and Growth

Engineering Managers conduct bi-annual performance reviews with each direct report. Reviews cover: delivery and quality of work, technical competency growth, collaboration and communication, and progress against professional development goals. Engineers are encouraged to maintain a personal development plan (PDP) in Confluence, updated quarterly, and shared with their Engineering Manager.

Section 40

Appendix

A. Glossary

Term	Definition
ADR	Architecture Decision Record — a short document capturing a significant technical decision and its rationale.
DORA Metrics	Deployment Frequency, Lead Time for Changes, Mean Time to Recover, and Change Failure Rate — standard engineering performance metrics.
HPA	HorizontalPodAutoscaler — a Kubernetes resource that automatically scales the number of Pod replicas based on resource utilisation.

LLM	Large Language Model — a machine learning model trained on large text corpora capable of generating and reasoning over natural language.
MTTR	Mean Time to Recover — the average time from incident detection to resolution.
OKR	Objectives and Key Results — a framework for setting and measuring organisational goals.
PDB	PodDisruptionBudget — a Kubernetes policy ensuring minimum Pod availability during voluntary disruptions.
PIR	Post-Incident Review — a structured review document produced after a significant production incident.
RAG	Retrieval-Augmented Generation — an AI architecture that combines vector search retrieval with LLM-based generation for grounded responses.
RBAC	Role-Based Access Control — an authorisation model where access permissions are assigned to roles, and users are assigned to roles.
SLO	Service Level Objective — a target metric for service availability or performance (e.g. 99.9% uptime).
SAST	Static Application Security Testing — automated code analysis for security vulnerabilities without executing the code.
SemVer	Semantic Versioning — a versioning scheme following MAJOR.MINOR.PATCH conventions.

TTL	Time-to-Live — a duration after which a cached entry or DNS record expires and must be refreshed.

B. Approved Technology Reference

Category	Technology	Use Case
Programming Language	Python 3.11+, TypeScript 5+, Java 21	Backend, ML integration, frontend
Backend Frameworks	FastAPI, Flask, Node.js / Express	REST API services
Frontend	React 18+, TypeScript, HTML5, CSS3	Web application UIs
Databases	PostgreSQL 16, Redis 7, MongoDB 7, pgvector	Relational, cache, document, vector storage
Cloud Infrastructure	AWS, Azure	Production and staging environments
Containerisation	Docker, Kubernetes	Service packaging and orchestration
CI/CD	GitHub Actions	Automated build, test, and deployment
Observability	Prometheus, Grafana, ELK Stack, OpenTelemetry	Metrics, logs, traces

Security Scanning	Trivy, Snyk, pip-audit, Semgrep	Dependency and code vulnerability scanning
AI Integration	LangChain, LlamaIndex, Ollama, Hugging Face	LLM and RAG pipeline development
API Testing	Postman, Newman, Pytest, Playwright	Contract, integration, and E2E testing
Project Management	Jira, Confluence	Issue tracking, documentation
Version Control	Git, GitHub Enterprise	Source code management
IaC	Terraform, Helm	Infrastructure provisioning
Web Server	Nginx	Reverse proxy, static file serving

C. Handbook Revision History

Revision	Date	Summary	Approved By
2025-1	June 2025	Initial publication of the Software Engineering Department Handbook.	Engineering Leadership

D. Key Contacts

Role	Contact Method
Engineering Leadership	`#engineering-leadership` Slack channel
Information Security	`#infosec` Slack channel; Jira InfoSec project for formal requests
IT Support	IT helpdesk portal; `#it-support` Slack for urgent issues
On-Call Incident Response	`#incidents` Slack channel; PagerDuty escalation policy
Handbook Feedback	`#engineering-standards` Slack; Confluence handbook-proposal label